ProveIt AI Interview Cycle - Code Walkthrough

Each section captures a code block, explains it, then moves to the next block.
Generated for the Zara recruiter-agent AI interview flow.

1. CV upload starts the AI cycle

Location: backend/src/controllers/aiInterviewController.js:62-76

```
 62 | export async function uploadCV(req, res) {
 63 |   try {
 64 |     if (!req.file) {
 65 |       return res.status(400).json({ msg: "No CV file uploaded" });
 66 |     }
 67 |
 68 |     if (req.file.mimetype !== "application/pdf") {
 69 |       return res.status(400).json({ msg: "Only PDF files are allowed" });
 70 |     }
 71 |
 72 |     // In mock mode we bypass PDF parsing entirely so production testing never
 73 |     // depends on pdf-parse or mupdf.
 74 |     let analysis;
 75 |     if (OPENAI_MOCK_MODE) {
 76 |       analysis = await analyzeCv("mock cv upload");
```

Explanation:
- Line 62 defines the uploadCV controller for POST /api/ai-interview/upload-cv.
- Lines 64-70 validate that a PDF file exists and reject invalid uploads early.
- Lines 75-76 switch to local mock CV analysis when OPENAI_MOCK_MODE is true.

2. CV text is extracted and analyzed

Location: backend/src/controllers/aiInterviewController.js:78-123

```
 78 |       // Extract text from the uploaded PDF buffer.
 79 |       let cvText = "";
 80 |       try {
 81 |         const parsed = await pdfParse(req.file.buffer);
 82 |         cvText = (parsed.text || "").trim();
 83 |       } catch (error) {
 84 |         // A parse failure here is not fatal: we can still try the image fallback.
 85 |         console.error("Failed to parse PDF text layer:", error.message);
 86 |       }
 87 |
 88 |       // Analyze the CV. Prefer the extracted text; if the PDF has no text layer
 89 |       // (scanned / image-based), fall back to rendering the pages to images and
 90 |       // reading them with gpt-4o-mini vision.
 91 |       if (cvText) {
 92 |         analysis = await analyzeCv(cvText);
 93 |       } else {
 94 |         try {
 95 |           const imageBuffers = await pdfToPngBuffers(req.file.buffer);
 96 |           analysis = await analyzeCvFromImages(imageBuffers);
 97 |         } catch (error) {
 98 |           console.error("Image-based CV analysis failed:", error.message);
 99 |           return res.status(400).json({
100 |             msg:
101 |               "We couldn’t read this PDF. Please make sure it is a valid CV and " +
102 |               "try again, ideally a PDF where the text can be selected.",
103 |           });
104 |         }
105 |       }
106 |     }
107 |
108 |     const cvSummary = [
109 |       ‘Primary field: ${analysis.primaryField}‘,
110 |       ‘Years of experience: ${analysis.yearsOfExperience}‘,
111 |       ‘Key skills: ${(analysis.keySkills || []).join(", ")}‘,
112 |       ‘Strongest areas: ${(analysis.strongestAreas || []).join(", ")}‘,
113 |       ‘Notable technologies: ${(analysis.notableTechnologies || []).join(", ")}‘,
114 |     ].join("\n");
115 |
116 |     const profilePayload = {
```

```
117 |       primaryField: analysis.primaryField,
118 |       yearsOfExperience: analysis.yearsOfExperience,
119 |       keySkills: analysis.keySkills || [],
120 |       strongestAreas: analysis.strongestAreas || [],
121 |       notableTechnologies: analysis.notableTechnologies || [],
122 |       cvSummary,
123 |     };
```

Explanation:

- Lines 79-82 extract selectable text from the PDF buffer.
- Lines 91-92 send extracted CV text to the AI CV analyzer.
- Lines 95-96 use image-based CV analysis if the PDF has no text layer.
- Lines 108-114 build the concise CV summary reused during the interview.
- Lines 116-123 prepare the profile payload saved for this user.

3. The CV analyzer calls the AI provider

Location: backend/src/lib/openai.js:329-356

```
329 | export async function analyzeCv(cvText) {
330 |   try {
331 |     if (MOCK_MODE) {
332 |       return buildMockAnalysis(cvText);
333 |     }
334 |
335 |     const systemPrompt = "You are an expert technical recruiter. You analyze software engineering
CVs and extract a concise structured summary. Respond with ONLY valid JSON and nothing else.";
336 |     const messages = [
337 |       {
338 |         role: "user",
339 |         content:
340 |           "Analyze the following CV and return a JSON object with this exact " +
341 |           "structure:\n" +
342 |           "{\n" +
343 |           '  "primaryField": string (one of: frontend, backend, fullstack, "data science", mobile,
devops),\n' +
344 |           '  "yearsOfExperience": number,\n' +
345 |           '  "keySkills": string[],\n' +
346 |           '  "strongestAreas": string[],\n' +
347 |           '  "notableTechnologies": string[]\n' +
348 |           "}\n\n" +
349 |           "Respond with ONLY valid JSON and nothing else.\n\n" +
350 |           "CV:\n" +
351 |           cvText,
352 |       }
353 |     ];
354 |
355 |     const responseData = await callITIModel(messages, systemPrompt, false);
356 |     return parseJsonResponse(responseData, "analyzing the CV");
```

Explanation:

- Line 329 defines analyzeCv, which turns CV text into structured profile data.
- Lines 331-333 return mock profile data when local mock mode is enabled.
- Lines 335-353 build a strict JSON prompt for the AI model.
- Line 355 calls the configured ITI provider.
- Line 356 parses the AI response as JSON.

4. Start Interview loads the candidate profile

Location: backend/src/controllers/aiInterviewController.js:223-250

```
223 | export async function startInterview(req, res) {
224 |   try {
225 |     const dbProfile = await CandidateProfile.findOne({ userId: req.user._id });
226 |     const mockProfile = OPENAI_MOCK_MODE ? req.body?.mockProfile || null : null;
227 |     const profile = dbProfile || mockProfile;
228 |     if (!profile) {
229 |       return res
230 |         .status(400)
231 |         .json({ msg: "Please upload your CV before starting an interview" });
232 |     }// Paywall (no-op until enabled).
233 |     const quota = await checkAndConsumeInterviewQuota(req.user._id);
```

```
234 |     if (!quota.canProceed) {
235 |       return res.status(402).json({
236 |         msg: quota.message,
237 |         upgradeRequired: true,
238 |       });
239 |     }
240 |
241 |     // Generate fresh, stack-specific spoken questions with GPT, biased toward
242 |     // the candidate's previously weak topics so the interview adapts over time.
243 |     const weakTopics = profile.topicWeaknesses
244 |       ? Object.keys(profile.topicWeaknesses)
245 |       : [];
246 |     const language = String(req.body.language || "en")
247 |       .toLowerCase()
248 |       .startsWith("ar")
249 |       ? "ar"
250 |       : "en";
```

Explanation:

- Line 223 defines startInterview for POST /api/ai-interview/start.
- Lines 225-227 load the saved CandidateProfile or mock profile.
- Lines 228-231 stop the flow if the user has not uploaded a CV.
- Lines 243-250 prepare weak topics and language for question generation.

5. Start Interview generates Zara questions

Location: backend/src/controllers/aiInterviewController.js:252-313

```
252 |     let selected;
253 |     const targetRole = String(req.body.targetRole || profile.primaryField || "");
254 |     const jobDescription = String(req.body.jobDescription || "");
255 |     try {
256 |       selected = await prepareInterviewQuestions({
257 |         profile,
258 |         weakTopics,
259 |         language,
260 |         targetRole,
261 |         jobDescription,
262 |       });
263 |     } catch (genError) {
264 |       console.error("Question generation failed:", genError.message);
265 |       return res
266 |         .status(500)
267 |         .json({ msg: "Could not prepare your interview, please try again" });
268 |     }
269 |
270 |     if (!selected || selected.length < MIN_QUESTIONS) {
271 |       return res.status(404).json({
272 |         msg: "No questions available, please contact support",
273 |       });
274 |     }
275 |
276 |     const session = await AIInterviewSession.create({
277 |       userId: req.user._id,
278 |       cvAnalysis: {
279 |         primaryField: profile.primaryField,
280 |         yearsOfExperience: profile.yearsOfExperience,
281 |         keySkills: profile.keySkills,
282 |         strongestAreas: profile.strongestAreas,
283 |         notableTechnologies: profile.notableTechnologies,
284 |         cvSummary: profile.cvSummary,
285 |       },
286 |       questions: selected.map((q) => ({
287 |         questionText: q.questionText,
288 |         topic: q.topic || "general",
289 |         difficulty: q.difficulty || "medium",
290 |         stage: q.stage || "technical",
291 |       })),
292 |       language,
293 |       interviewerName: "Zara",
294 |       currentStage: selected[0]?.stage || "intro",
295 |       targetRole,
296 |       jobDescription,
297 |       stageHistory: [
298 |         {
```

```
299 |           stage: selected[0]?.stage || "intro",
300 |           questionIndex: 0,
301 |           enteredAt: new Date(),
302 |         },
303 |       ],
304 |       status: "in-progress",
305 |     });
306 |     const firstMeta = buildQuestionResponseMeta(session.questions[0], 0);
307 |
308 |     res.status(201).json({
309 |       sessionId: session._id,
310 |       totalQuestions: session.questions.length,
311 |       firstQuestion: session.questions[0].questionText,
312 |       ...firstMeta,
313 |     });
```

Explanation:
- Lines 253-254 read optional target role and job description.
- Lines 255-262 call prepareInterviewQuestions to generate staged questions.
- Lines 276-305 create the AIInterviewSession document.
- Lines 286-291 store question text, topic, difficulty, and stage.
- Lines 308-313 return the first question and Zara stage metadata to the frontend.

6. Service layer normalizes each generated question

Location: backend/src/ai/aiInterviewService.js:11-37

```
 11 | export async function prepareInterviewQuestions({
 12 |   profile,
 13 |   weakTopics = [],
 14 |   language = "en",
 15 |   jobDescription = "",
 16 |   targetRole = "",
 17 | } = {}) {
 18 |   const generated = await generateInterviewQuestions({
 19 |     cvSummary: profile?.cvSummary,
 20 |     primaryField: profile?.primaryField,
 21 |     weakTopics,
 22 |     language,
 23 |     jobDescription,
 24 |     targetRole,
 25 |   });
 26 |
 27 |   return generated.map((question, index) => {
 28 |     const stage = getStageForQuestion(index, question.stage);
 29 |
 30 |     return {
 31 |       questionText: question.questionText,
 32 |       topic: question.topic || stage,
 33 |       difficulty: question.difficulty || "medium",
 34 |       stage,
 35 |     };
 36 |   });
 37 | }
```

Explanation:
- Lines 11-17 define the clean service API for preparing questions.
- Lines 18-25 call generateInterviewQuestions with CV, role, JD, weak topics, and language.
- Line 28 resolves the interview stage for each question.
- Lines 30-35 normalize the final question object used by the session.

7. Question generation uses AI, then falls back safely

Location: backend/src/lib/openai.js:419-520

```
419 | export async function generateInterviewQuestions({
420 |   cvSummary,
421 |   primaryField,
422 |   weakTopics = [],
423 |   language = "en",
424 |   jobDescription = "",
425 |   targetRole = "",
```

```
 426 | } = {}) {
 427 |   try {
 428 |     if (MOCK_MODE) {
 429 |       return buildMockQuestions(primaryField, weakTopics, language);
 430 |     }
 431 |
 432 |     const weakTopicsLine =
 433 |       weakTopics && weakTopics.length > 0
 434 |         ? ‘The candidate has previously been weaker in these topics, so ask MORE questions about
them: ${weakTopics.join(", ")}.\n\n‘
 435 |         : "";
 436 |
 437 |     const arabic = isArabicLanguage(language);
 438 |     const systemPrompt = arabic
 439 |       ? "??? ????? ???? ????. ??? ????? ????? ?????? ????? ?????? ???????. ??? ?? ???? ???????
?????? ??????? ????? ??????? ??? ???? ?? ?????? ????? ?????. ??? ??? ????? JSON ?????."
 440 |       : "You are an expert technical interviewer preparing a SPOKEN interview. You generate
up-to-date, relevant questions tailored to the candidate’s tech stack using your own knowledge. The
questions are read aloud and answered by voice, so they must be conversational and must NEVER ask the
candidate to write or type code. Respond with ONLY valid JSON and nothing else.";
 441 |
 442 |     const messages = [
 443 |       {
 444 |         role: "user",
 445 |         content: arabic
 446 |           ? ‘?????? ??????? ??????: ${primaryField}\n\n‘ +
 447 |             ‘???????:\n${cvSummary}\n\n‘ +
 448 |             weakTopicsLine +
 449 |             "???? ??? 5 ?7 ????? ?????? ????? ?????? ???? ??????? ?? ?????? ??? ??????. ???? JSON
??????? ??????:\n" +
 450 |             "{\n" +
 451 |             ’  "questions": [\n’ +
 452 |             "    {\n" +
 453 |             ’      "questionText": string,\n’ +
 454 |             ’      "topic": string,\n’ +
 455 |             ’      "difficulty": string (one of: easy, medium, hard)\n’ +
 456 |             "    }\n" +
 457 |             "  ]\n" +
 458 |             "}\n\n" +
 459 |             "??? ??? ????? JSON ?????."
 460 |           : ‘Candidate primary field: ${primaryField}\n\n‘ +
 461 |             ‘Candidate background:\n${cvSummary}\n\n‘ +
 462 |             weakTopicsLine +
 463 |             "Generate between 5 and 7 spoken interview questions tailored to " +
 464 |             "this candidate’s tech stack and experience, ramping from easier " +
 465 |             "to harder. Return a JSON object with this exact structure:\n" +
 466 |             "{\n" +
 467 |             ’  "questions": [\n’ +
 468 |             "    {\n" +
 469 |             ’      "questionText": string (a spoken question, no code writing),\n’ +
 470 |             ’      "topic": string (a short kebab-case tag, e.g. "react-hooks", "system-design",
"sql-indexing"),\n’ +
 471 |             ’      "difficulty": string (one of: easy, medium, hard)\n’ +
 472 |             "    }\n" +
 473 |             "  ]\n" +
 474 |             "}\n\n" +
 475 |             "Respond with ONLY valid JSON and nothing else.",
 476 |       }
 477 |     ];
 478 |
 479 |     const responseData = await callITIModel(
 480 |       [
 481 |         {
 482 |           role: "user",
 483 |           content: buildQuestionPlanPrompt({
 484 |             cvSummary,
 485 |             primaryField,
 486 |             weakTopics,
 487 |             language,
 488 |             jobDescription,
 489 |             targetRole,
 490 |           }),
 491 |         },
 492 |       ],
 493 |       buildZaraSystemPrompt(language),
 494 |       false,
```

```
495 |     );
496 |     const parsed = parseJsonResponse(responseData, "generating interview questions");
497 |     const questions = Array.isArray(parsed) ? parsed : parsed.questions;
498 |
499 |     const normalizedQuestions = (questions || [])
500 |       .filter((q) => q && q.questionText)
501 |       .map((q) => ({
502 |         questionText: String(q.questionText),
503 |         topic: String(q.topic || "general"),
504 |         difficulty: String(q.difficulty || "medium").toLowerCase(),
505 |         stage: String(q.stage || ""),
506 |       }))
507 |       .slice(0, 7);
508 |
509 |     if (normalizedQuestions.length < 5) {
510 |       console.error(
511 |         "ITI Gateway returned too few interview questions; using local Zara fallback.",
512 |       );
513 |       return buildMockQuestions(primaryField, weakTopics, language);
514 |     }
515 |
516 |     return normalizedQuestions;
517 |   } catch (error) {
518 |     console.error("generateInterviewQuestions error:", error.message);
519 |     return buildMockQuestions(primaryField, weakTopics, language);
520 |   }
```

Explanation:

- Lines 419-426 define generateInterviewQuestions inputs.
- Lines 428-430 immediately use local mock questions when mock mode is true.
- Lines 479-495 call the AI provider with Zara’s staged recruiter prompt.
- Lines 496-507 parse and normalize the model’s question list.
- Lines 509-514 fall back if the model returns fewer than five questions.
- Lines 517-519 fall back if the AI provider throws an error.

8. ITI provider reads env variables and calls the gateway

Location: backend/src/ai/providers/itiProvider.js:7-44

```
 7 | export function getItiConfig({ isVision = false } = {}) {
 8 |   return {
 9 |     baseUrl: ENV.ITI_API_BASE_URL || DEFAULT_BASE_URL,
10 |     apiKey: ENV.ITI_API_KEY || ENV.OPENAI_API_KEY,
11 |     model: isVision
12 |       ? ENV.ITI_VISION_MODEL || DEFAULT_VISION_MODEL
13 |       : ENV.ITI_MODEL || DEFAULT_MODEL,
14 |   };
15 | }
16 |
17 | export async function callItiProvider(messages, systemPrompt, options = {}) {
18 |   const { baseUrl, apiKey, model } = getItiConfig(options);
19 |   const selectedModel = options.model || model;
20 |
21 |   if (!apiKey) {
22 |     throw new Error("ITI API key is not configured");
23 |   }
24 |
25 |   const response = await fetch(baseUrl, {
26 |     method: "POST",
27 |     headers: {
28 |       Authorization: `Bearer ${apiKey}`,
29 |       "Content-Type": "application/json",
30 |     },
31 |     body: JSON.stringify({
32 |       model_id: selectedModel,
33 |       messages,
34 |       system_prompt: systemPrompt || "",
35 |     }),
36 |   });
37 |
38 |   if (!response.ok) {
39 |     const errorText = await response.text().catch(() => "");
40 |     throw new Error(`ITI Gateway Error: ${response.status} - ${errorText}`);
41 |   }
```

```
42 |
43 |   return response.json();
44 | }
```

Explanation:
- Lines 7-15 read ITI_API_BASE_URL, ITI_API_KEY, ITI_MODEL, and vision model config.
- Lines 17-19 choose the selected model for this request.
- Lines 21-23 fail fast if no API key is configured.
- Lines 25-36 send the POST request to the ITI Gateway.
- Lines 38-40 throw a useful error when the gateway rejects the request.
- Line 43 returns the gateway JSON response.

9. Interview stages define Zara’s structure

Location: backend/src/ai/interviewStateMachine.js:1-31

```
 1 | export const INTERVIEW_STAGES = [
 2 |   "intro",
 3 |   "cv_review",
 4 |   "jd_review",
 5 |   "behavioral",
 6 |   "technical",
 7 |   "coding_challenge",
 8 |   "evaluation",
 9 |   "final_feedback",
10 | ];
11 |
12 | export const STAGE_LABELS = {
13 |   intro: "Interview setup",
14 |   cv_review: "CV review",
15 |   jd_review: "Role alignment",
16 |   behavioral: "Behavioral screen",
17 |   technical: "Technical depth",
18 |   coding_challenge: "Problem solving",
19 |   evaluation: "Answer evaluation",
20 |   final_feedback: "Final feedback",
21 | };
22 |
23 | const QUESTION_STAGES = [
24 |   "intro",
25 |   "cv_review",
26 |   "jd_review",
27 |   "behavioral",
28 |   "technical",
29 |   "technical",
30 |   "coding_challenge",
31 | ];
```

Explanation:
- Lines 1-10 define the full stage list.
- Lines 12-21 define human-readable labels for the UI.
- Lines 23-31 map question order to intro, CV, role, behavioral, technical, and problem-solving stages.

10. Stage metadata is returned to the frontend

Location: backend/src/ai/interviewStateMachine.js:33-83

```
33 | export function normalizeStage(stage) {
34 |   return INTERVIEW_STAGES.includes(stage) ? stage : "intro";
35 | }
36 |
37 | export function getStageForQuestion(index, explicitStage) {
38 |   if (explicitStage && INTERVIEW_STAGES.includes(explicitStage)) {
39 |     return explicitStage;
40 |   }
41 |
42 |   return QUESTION_STAGES[Math.min(Number(index) || 0, QUESTION_STAGES.length - 1)];
43 | }
44 |
45 | export function getStageLabel(stage) {
46 |   return STAGE_LABELS[normalizeStage(stage)];
```

```
47 | }
48 |
49 | export function getStageTransition(stage) {
50 |   const normalized = normalizeStage(stage);
51 |
52 |   const transitions = {
53 |     intro:
54 |       "I will start by setting context, then we will move through your background and role fit.",
55 |     cv_review:
56 |       "Now I want to connect the interview to what you have actually done.",
57 |     jd_review:
58 |       "Next, I will check how your experience maps to the target role.",
59 |     behavioral:
60 |       "Let’s shift into recruiter-style behavioral signals.",
61 |     technical:
62 |       "Now we will go deeper technically and keep the questions role-based.",
63 |     coding_challenge:
64 |       "I will use a short problem-solving prompt to understand your approach.",
65 |     evaluation:
66 |       "I am reviewing your answer before deciding the next best question.",
67 |     final_feedback:
68 |       "I am preparing a structured summary with strengths, gaps, and readiness.",
69 |   };
70 |
71 |   return transitions[normalized];
72 | }
73 |
74 | export function buildStageMeta(stage) {
75 |   const normalized = normalizeStage(stage);
76 |
77 |   return {
78 |     stage: normalized,
79 |     stageLabel: getStageLabel(normalized),
80 |     transitionMessage: getStageTransition(normalized),
81 |     interviewerName: "Zara",
82 |   };
83 | }
```

Explanation:

- Lines 33-35 validate stage names.
- Lines 37-43 choose an explicit stage or stage-by-question index.
- Lines 49-72 define transition messages Zara displays between stages.
- Lines 74-83 package stage, label, transition message, and interviewerName.

11. Frontend initializes the interview session UI

Location: frontend/src/pages/InterviewPage.jsx:33-67

```
33 | function InterviewPage() {
34 |   const navigate = useNavigate();
35 |   const { sessionId } = useParams();
36 |   const { state } = useLocation();
37 |   const { language: appLanguage } = useLanguage();
38 |
39 |   const submitMutation = useSubmitAnswer();
40 |   const [interviewLanguage, setInterviewLanguage] = useState(
41 |     state?.language || appLanguage || "en",
42 |   );
43 |   const t = getTranslator(interviewLanguage || appLanguage);
44 |   const { speak, cancel: cancelSpeech, isSpeaking } = useSpeechSynthesis({
45 |     language: interviewLanguage,
46 |   });
47 |
48 |   // Interview state.
49 |   const [questionIndex, setQuestionIndex] = useState(0);
50 |   const [totalQuestions, setTotalQuestions] = useState(
51 |     state?.totalQuestions || 0,
52 |   );
53 |   const [currentQuestion, setCurrentQuestion] = useState(
54 |     state?.firstQuestion || "",
55 |   );
56 |   const [status, setStatus] = useState("speaking");
57 |   const [isTypingMode, setIsTypingMode] = useState(false);
58 |   const [isFollowUp, setIsFollowUp] = useState(false);
59 |   const [loading, setLoading] = useState(!state?.firstQuestion);
```

```
  const [stage, setStage] = useState(state?.stage || "intro");
  const [stageLabel, setStageLabel] = useState(
    state?.stageLabel || getStageLabel(state?.stage || "intro"),
  );
  const [transitionMessage, setTransitionMessage] = useState(
    state?.transitionMessage || t("aiInterview.defaultTransition"),
  );
  const interviewerName = state?.interviewerName || "Zara";
```

Explanation:
- Line 33 defines the InterviewPage component.
- Lines 34-37 read navigation, session id, route state, and app language.
- Lines 39-46 initialize the answer mutation, translator, and browser speech synthesis.
- Lines 49-67 store current question, status, stage, transition message, and interviewer name.

12. Browser speech recognition captures candidate answer

Location: frontend/src/pages/InterviewPage.jsx:82-97

```
  // Voice recognition: auto-submit after the silence window.
  const {
    isSupported,
    isListening,
    transcript,
    setTranscript,
    start: startListening,
    stop: stopListening,
    reset: resetTranscript,
  } = useSpeechRecognition({
    silenceTimeout: 2500,
    language: interviewLanguage,
    onSilence: (finalText) => {
      if (finalText && finalText.trim()) submitAnswer(finalText);
    },
  });
```

Explanation:
- Lines 82-91 initialize the browser SpeechRecognition wrapper.
- Line 92 sets the silence timeout.
- Line 93 sets the recognition language.
- Lines 94-96 auto-submit final speech text when the candidate stops speaking.

13. Frontend submits each answer

Location: frontend/src/pages/InterviewPage.jsx:101-166

```
  const submitAnswer = useCallback(
    (answerText, { skipped = false } = {}) => {
      if (submittingRef.current) return;
      submittingRef.current = true;

      setStatus("processing");

      submitMutation.mutate(
        {
          sessionId,
          questionIndex,
          answer: answerText || "",
          timeSpent: timeSpentRef.current,
          isFollowUp,
          skipped,
        },
        {
          onSuccess: (data) => {
            submittingRef.current = false;
            timeSpentRef.current = 0;
            resetTranscript();
            setTranscript("");

            if (data.askFollowUp && data.followUpQuestion) {
              setIsFollowUp(true);
              setCurrentQuestion(data.followUpQuestion);
              setStage(data.stage || stage);
```

```
128 |               setStageLabel(data.stageLabel || stageLabel);
129 |               setTransitionMessage(data.transitionMessage || transitionMessage);
130 |               setStatus("speaking");
131 |               return;
132 |             }
133 |
134 |             if (data.isComplete) {
135 |               finishInterview();
136 |               return;
137 |             }
138 |
139 |             setIsFollowUp(false);
140 |             setQuestionIndex(data.questionIndex);
141 |             setCurrentQuestion(data.nextQuestion);
142 |             setStage(data.stage || stage);
143 |             setStageLabel(data.stageLabel || stageLabel);
144 |             setTransitionMessage(data.transitionMessage || transitionMessage);
145 |             setStatus("speaking");
146 |           },
147 |           onError: () => {
148 |             submittingRef.current = false;
149 |             setStatus("listening");
150 |           },
151 |         },
152 |       );
153 |     },
154 |     [
155 |       finishInterview,
156 |       isFollowUp,
157 |       questionIndex,
158 |       resetTranscript,
159 |       sessionId,
160 |       stage,
161 |       stageLabel,
162 |       setTranscript,
163 |       submitMutation,
164 |       transitionMessage,
165 |     ],
166 |   );
```

Explanation:

- Lines 101-103 define submitAnswer and prevent duplicate submits.
- Line 106 switches the UI to processing.
- Lines 108-116 send the answer payload to the backend.
- Lines 124-131 handle adaptive follow-up questions.
- Lines 134-136 navigate to results when complete.
- Lines 139-145 move to the next main question and update stage metadata.

14. Browser speech synthesis reads Zara’s question aloud

Location: frontend/src/pages/InterviewPage.jsx:212-227

```
212 |   // Speak each new question, then start listening (voice mode only).
213 |   useEffect(() => {
214 |     if (!currentQuestion || status !== "speaking") return;
215 |
216 |     if (isTypingMode || !isSupported) {
217 |       setStatus("listening");
218 |       return;
219 |     }
220 |
221 |     resetTranscript();
222 |     speak(currentQuestion, () => {
223 |       setStatus("listening");
224 |       startListening();
225 |     });
226 |     // eslint-disable-next-line react-hooks/exhaustive-deps
227 |   }, [currentQuestion, status, isTypingMode, isSupported, speak, startListening]);
```

Explanation:

- Lines 213-214 only run when there is a question and Zara is speaking.
- Lines 216-219 skip voice when typing mode or unsupported browser is used.
- Line 221 clears old transcript.

- Lines 222-225 speak the question, then start listening.

15. Browser voice implementation

Location: frontend/src/hooks/useSpeechSynthesis.js:10-80

```
10 | export default function useSpeechSynthesis({ language = "en" } = {}) {
11 |   const isSupported =
12 |     typeof window !== "undefined" && "speechSynthesis" in window;
13 |
14 |   const [isSpeaking, setIsSpeaking] = useState(false);
15 |   const voiceRef = useRef(null);
16 |
17 |   // Resolve the preferred voice now and again when the list loads.
18 |   useEffect(() => {
19 |     if (!isSupported) return;
20 |
21 |     const loadVoice = () => {
22 |       const voices = window.speechSynthesis?.getVoices?.() || [];
23 |       const isArabic = String(language).toLowerCase().startsWith("ar");
24 |       const preferredPrefix = isArabic ? "ar" : "en";
25 |       voiceRef.current =
26 |         voices.find((v) => v.lang?.toLowerCase().startsWith(preferredPrefix)) ||
27 |         voices.find((v) => v.lang?.toLowerCase().startsWith("en")) ||
28 |         voices[0] ||
29 |         null;
30 |     };
31 |     loadVoice();
32 |     window.speechSynthesis.addEventListener("voiceschanged", loadVoice);
33 |
34 |     return () => {
35 |       window.speechSynthesis.removeEventListener("voiceschanged", loadVoice);
36 |       window.speechSynthesis.cancel();
37 |     };
38 |   }, [isSupported, language]);
39 |
40 |   const cancel = useCallback(() => {
41 |     if (!isSupported) return;
42 |     window.speechSynthesis.cancel();
43 |     setIsSpeaking(false);
44 |   }, [isSupported]);
45 |
46 |   // Speak text, invoking onEnd when finished (or immediately if unsupported).
47 |   const speak = useCallback(
48 |     (text, onEnd) => {
49 |       if (!isSupported || !text) {
50 |         onEnd?.();
51 |         return;
52 |       }
53 |
54 |       // Always cancel ongoing speech first to avoid overlap.
55 |       window.speechSynthesis.cancel();
56 |
57 |       const utterance = new SpeechSynthesisUtterance(text);
58 |       if (voiceRef.current) utterance.voice = voiceRef.current;
59 |       utterance.lang =
60 |         voiceRef.current?.lang ||
61 |         (String(language).toLowerCase().startsWith("ar") ? "ar-SA" : "en-US");
62 |       utterance.rate = 1;
63 |       utterance.pitch = 1;
64 |
65 |       utterance.onstart = () => setIsSpeaking(true);
66 |       utterance.onend = () => {
67 |         setIsSpeaking(false);
68 |         onEnd?.();
69 |       };
70 |       utterance.onerror = () => {
71 |         setIsSpeaking(false);
72 |         onEnd?.();
73 |       };
74 |
75 |       window.speechSynthesis.speak(utterance);
76 |     },
77 |     [isSupported, language],
78 |   );
79 |
```

```
  80 |   return { speak, cancel, isSpeaking, isSupported };
```

Explanation:
- Lines 10-13 check browser SpeechSynthesis support.
- Lines 21-30 choose a voice matching the interview language.
- Lines 47-52 define the speak function and handle unsupported browsers.
- Lines 57-63 create the browser utterance and set language, rate, and pitch.
- Line 75 calls window.speechSynthesis.speak, so Zara’s sound is from the browser.

16. Backend evaluates answers and returns follow-ups

Location: backend/src/controllers/aiInterviewController.js:324-417

```
 324 | export async function submitAnswer(req, res) {
 325 |   try {
 326 |     const { sessionId } = req.params;
 327 |     const {
 328 |       questionIndex,
 329 |       answer = "",
 330 |       timeSpent = 0,
 331 |       isFollowUp = false,
 332 |       skipped = false,
 333 |     } = req.body;
 334 |
 335 |     const session = await AIInterviewSession.findOne({
 336 |       _id: sessionId,
 337 |       userId: req.user._id,
 338 |     });
 339 |
 340 |     if (!session) return res.status(404).json({ msg: "Session not found" });
 341 |     if (session.status === "completed") {
 342 |       return res.status(400).json({ msg: "Interview already completed" });
 343 |     }
 344 |
 345 |     const index = Number(questionIndex);
 346 |     const current = session.questions[index];
 347 |     if (!current) {
 348 |       return res.status(400).json({ msg: "Invalid question index" });
 349 |     }
 350 |
 351 |     // Skipped question: zero score, no GPT call, no follow-up.
 352 |     if (skipped) {
 353 |       current.userAnswer = "";
 354 |       current.aiFeedback = "Question skipped.";
 355 |       current.score = 0;
 356 |       current.timeSpent = Number(timeSpent) || 0;
 357 |       current.followUpAsked = false;
 358 |       await session.save();
 359 |     } else if (isFollowUp) {
 360 |       // Store the follow-up answer against the same question (no re-scoring).
 361 |       current.followUpAnswer = answer;
 362 |       current.timeSpent += Number(timeSpent) || 0;
 363 |       await session.save();
 364 |     } else {
 365 |       // Main answer: evaluate with GPT.
 366 |       const cvSummary = session.cvAnalysis?.cvSummary || "";
 367 |       const userField = session.cvAnalysis?.primaryField || "fullstack";
 368 |
 369 |       const evaluation = await evaluateInterviewAnswer({
 370 |         question: current,
 371 |         answer,
 372 |         cvSummary,
 373 |         userField,
 374 |         language: session.language || "en",
 375 |       });
 376 |
 377 |       current.userAnswer = answer;
 378 |       current.aiFeedback = evaluation.feedback || "";
 379 |       current.score = Math.max(0, Math.min(10, Number(evaluation.score) || 0));
 380 |       current.timeSpent = Number(timeSpent) || 0;
 381 |       current.followUpAsked = !!evaluation.askFollowUp;
 382 |       session.currentStage = current.stage || session.currentStage;
 383 |       await session.save();
 384 |
```

```
385 |       if (evaluation.askFollowUp && evaluation.followUpQuestion) {
386 |         const meta = buildQuestionResponseMeta(current, index);
387 |         return res.status(200).json({
388 |           askFollowUp: true,
389 |           followUpQuestion: evaluation.followUpQuestion,
390 |           questionIndex: index,
391 |           isComplete: false,
392 |           ...meta,
393 |         });
394 |       }
395 |     }
396 |
397 |     // Move to the next main question, if any.
398 |     const nextIndex = index + 1;
399 |     if (nextIndex < session.questions.length) {
400 |       const nextQuestion = session.questions[nextIndex];
401 |       const meta = buildQuestionResponseMeta(nextQuestion, nextIndex);
402 |       session.currentStage = meta.stage;
403 |       session.stageHistory = session.stageHistory || [];
404 |       session.stageHistory.push({
405 |         stage: meta.stage,
406 |         questionIndex: nextIndex,
407 |         enteredAt: new Date(),
408 |       });
409 |       await session.save();
410 |
411 |       return res.status(200).json({
412 |         askFollowUp: false,
413 |         isComplete: false,
414 |         questionIndex: nextIndex,
415 |         nextQuestion: nextQuestion.questionText,
416 |         ...meta,
417 |       });
```

Explanation:
- Lines 324-333 receive the answer payload.
- Lines 335-348 load and validate the session and question.
- Lines 352-358 handle skipped questions without AI.
- Lines 365-375 evaluate the main answer with AI.
- Lines 377-383 save feedback, score, timing, and stage.
- Lines 385-393 return a follow-up question if needed.
- Lines 397-417 return the next main question and stage metadata.

17. Final feedback and recommendation

Location: backend/src/controllers/aiInterviewController.js:460-510

```
460 | export async function getResults(req, res) {
461 |   try {
462 |     const { sessionId } = req.params;
463 |
464 |     const session = await AIInterviewSession.findOne({
465 |       _id: sessionId,
466 |       userId: req.user._id,
467 |     });
468 |
469 |     if (!session) return res.status(404).json({ msg: "Session not found" });
470 |
471 |     // If already finalized, just return it (idempotent).
472 |     if (session.status === "completed") {
473 |       return res.status(200).json({ session });
474 |     }
475 |
476 |     const scorecard = await buildFinalInterviewFeedback(session);
477 |
478 |     // Trust our own per-question average for overallScore rather than GPT's,
479 |     // so it always matches the displayed per-question scores.
480 |     const scored = session.questions.filter((q) => typeof q.score === "number");
481 |     const computedAverage =
482 |       scored.length > 0
483 |         ? scored.reduce((sum, q) => sum + q.score, 0) / scored.length
484 |         : 0;
485 |
```

```
486 |     session.overallScore = Math.round(computedAverage * 10) / 10;
487 |     session.overallFeedback = scorecard.overallFeedback || "";
488 |     session.strengths = scorecard.strengths || [];
489 |     session.weaknesses = scorecard.weaknesses || [];
490 |     session.recommendation = scorecard.recommendation || "";
491 |     session.reportSummary = scorecard.reportSummary || "";
492 |     session.currentStage = "final_feedback";
493 |     session.status = "completed";
494 |     await session.save();
495 |
496 |     // Fire the background progress-tracking job (Feature 6).
497 |     try {
498 |       await inngest.send({
499 |         name: "ai-interview/completed",
500 |         data: { sessionId: session._id.toString() },
501 |       });
502 |     } catch (sendError) {
503 |       // Don't fail the request if the event can't be queued.
504 |       console.error(
505 |         "Failed to send ai-interview/completed event:",
506 |         sendError.message,
507 |       );
508 |     }
509 |
510 |     res.status(200).json({ session });
```

Explanation:

- Line 460 defines getResults for the final scorecard.
- Lines 464-474 load the session and keep the endpoint idempotent.
- Line 476 asks AI to build final interview feedback.
- Lines 480-486 compute the average score from saved per-question scores.
- Lines 487-493 save final feedback, strengths, weaknesses, recommendation, and status.
- Line 510 returns the completed session to the frontend.